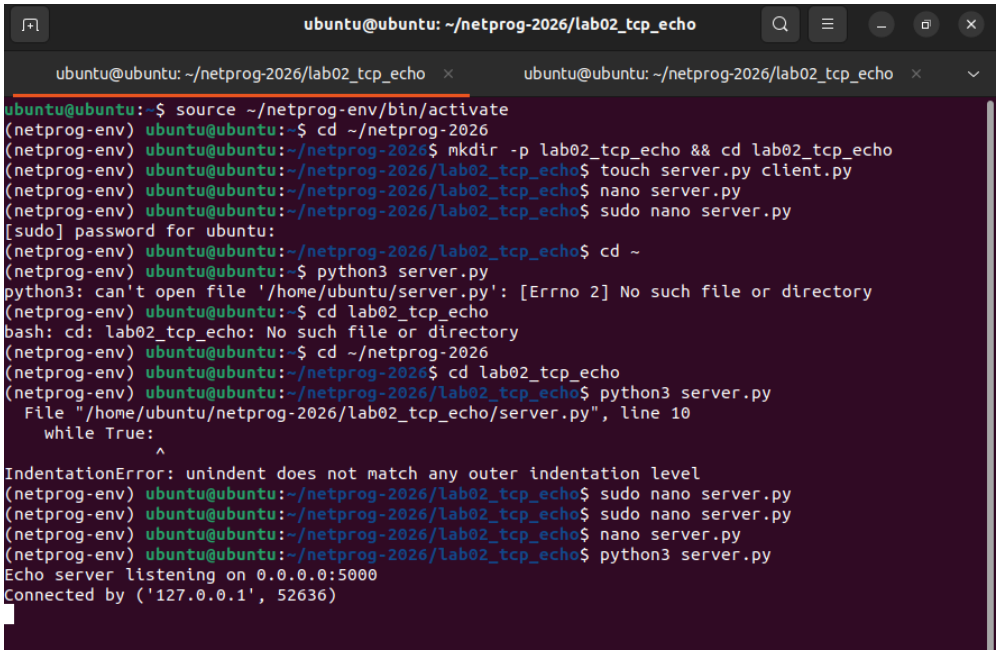
LAB 3

TCP Socket Programming: Echo Client-Server

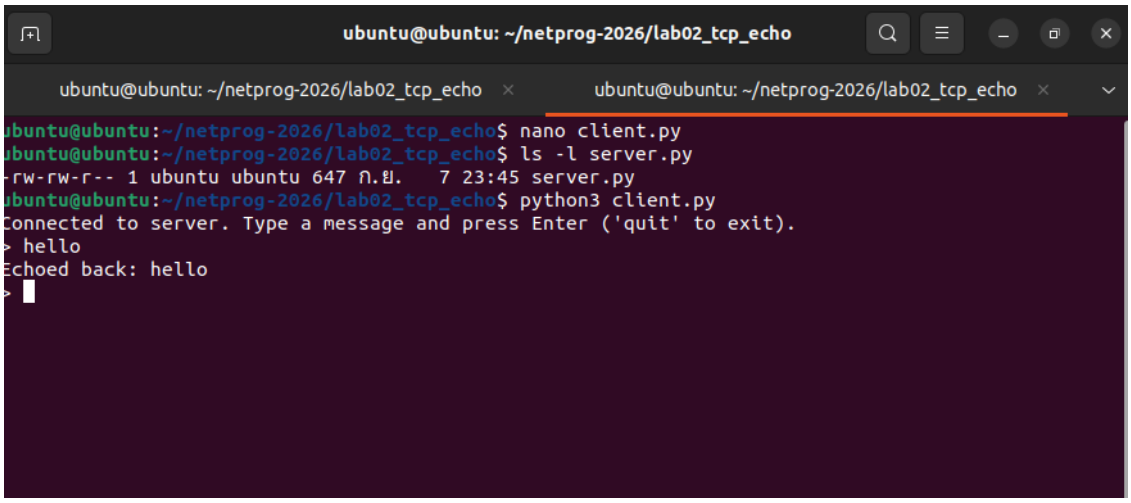
Lab Task

Task 1: Screenshot of server.py running plus a client sending "hello" and receiving "Echoed back: hello".

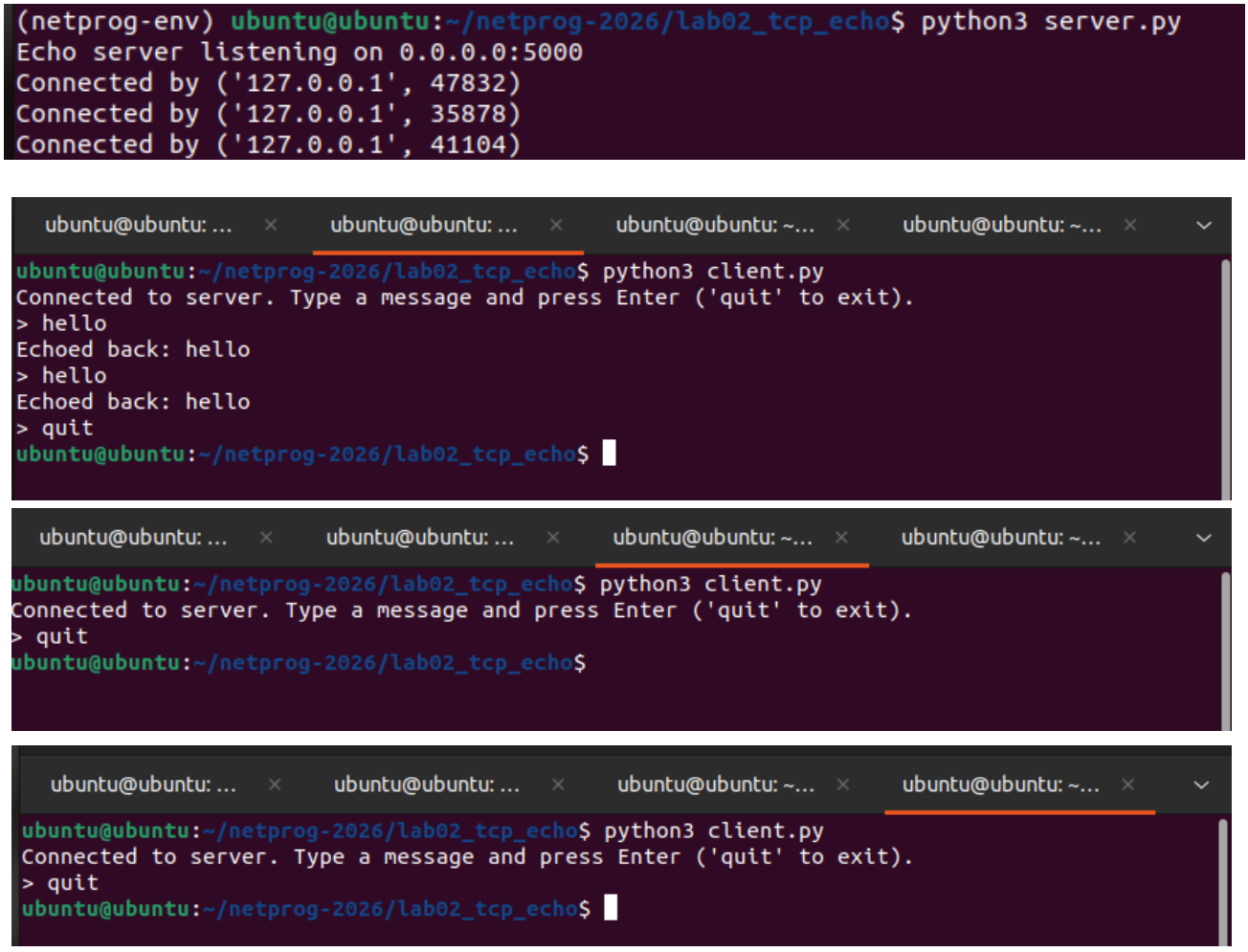
Terminal 1: server.py



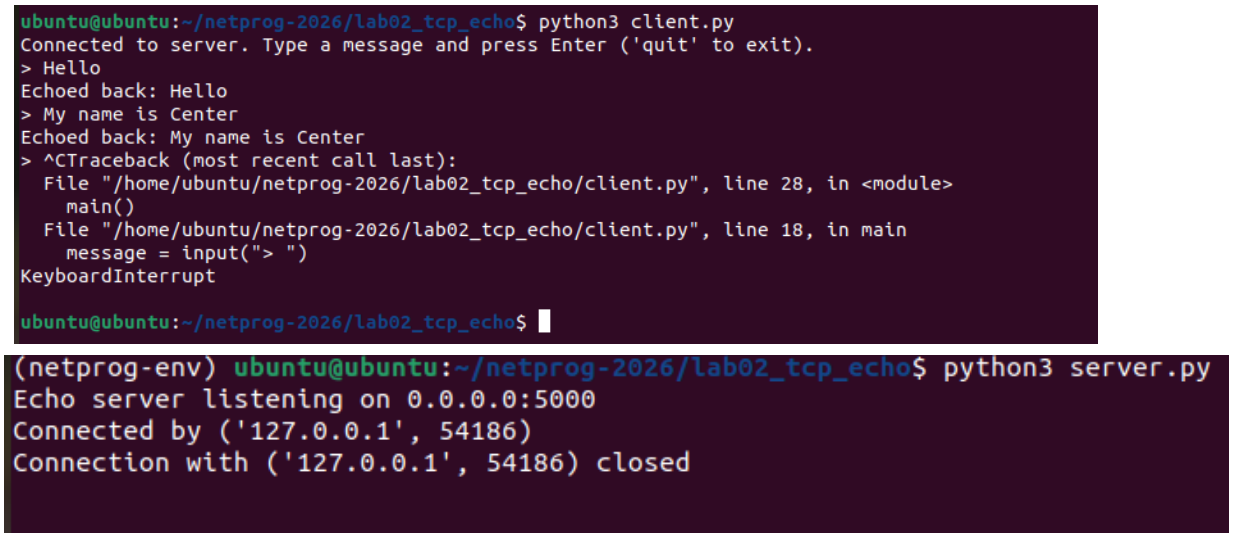
Terminal 2: client.py



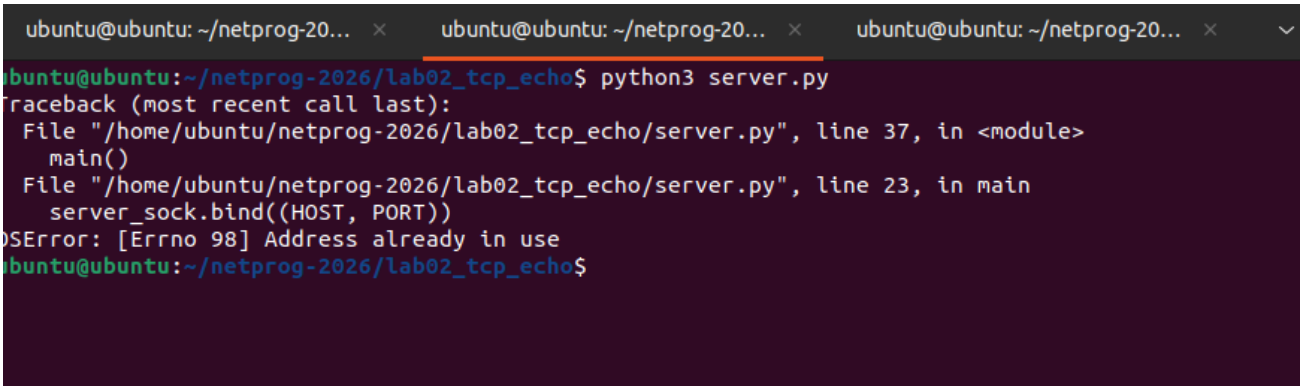
Task 2: Screenshot showing three simultaneous clients connected to the threaded server (Part E), each sending and receiving independently.



Task 3: Screenshot showing an abrupt client disconnect (Ctrl+C mid-conversation) with the server printing a clean message and continuing to run, not crashing.

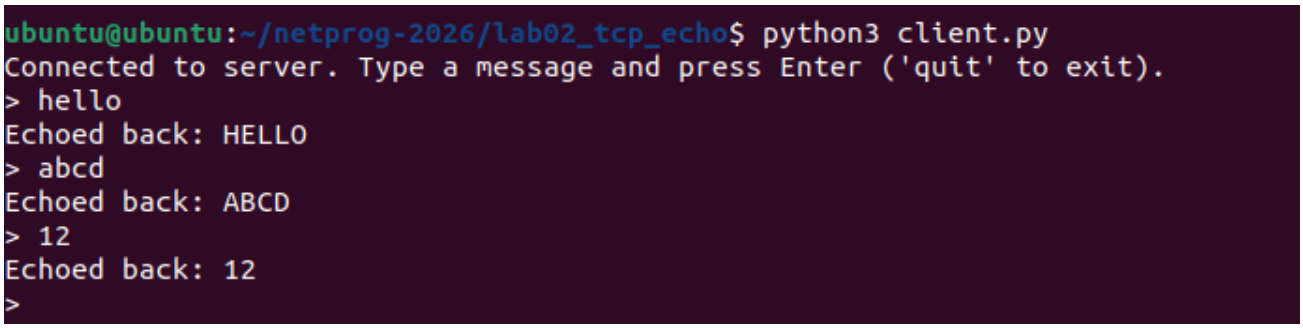


Task 4: Start a second instance of server.py while the first is still running on port 5000. Screenshot the resulting error, and in 1-2 sentences explain what caused it.



Answer a socket is already bound to port 5000, and TCP won't let a second process bind the same address/port pair while it's in use.

Task 5: Modify the server so it converts each echoed message to uppercase before sending it back (one-line change to handle_client). Screenshot the client receiving an uppercased echo.



Task 6: In 2-3 sentences, explain why recv(1024) is not guaranteed to return exactly the bytes one send() call sent, and why sendall() (rather than send()) is the safer choice for larger payloads.

Answer recv(1024) returns *whatever bytes have currently arrived*, up to 1024. TCP is a byte stream with no message boundaries, so a single send() can arrive split across multiple recv() calls, or multiple sends can coalesce into one recv(). sendall() is safer than send() because send() isn't guaranteed to transmit every byte in one call (it returns how many bytes it actually sent); sendall() loops internally until the entire buffer is flushed.

Reflection Questions

Answer briefly (2-4 sentences each) in your report:

1. Explain blocking I/O in the context of accept() and recv(). What exactly is the server thread doing (from the OS's point of view) while it waits?

Answer Blocking I/O means the program stops and waits until something happens, instead of moving on. When the server calls accept(), it waits until a client connects. When it calls recv(), it waits until data arrives. From the OS's point of view, the thread is sleeping. It uses no CPU time and is not doing any work. The OS simply wakes the thread up again once a connection or data is ready.

2. Why is threading (or an equivalent concurrency mechanism) needed to serve multiple clients with this blocking socket API? What would you observe if you removed the threading from Part E?

Answer Since accept() and recv() block, one thread can only handle one client's conversation at a time. Threading lets the server create a new thread for each client, so each one can wait for its own data without blocking the others. If we removed threading, the server would go back to the Part D problem: only the first client could send and receive messages, while every other client would hang and wait until the first one closed its connection.

3. Name a real-world application-layer protocol built on top of TCP sockets similar to what you built today and briefly describe one similarity and one difference from your echo protocol.

Answer HTTP is a protocol built on TCP sockets, similar to our echo protocol. Like echo server, HTTP uses connect(), send(), and recv() over a TCP connection between a client and a server. The difference is that our echo protocol just sends back the same bytes it receives, while HTTP uses a structured format like a request line, headers, and a body so the server can understand what the client is asking for and respond differently, such as returning a webpage.